



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления (ИУ)»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии (ИУ7)»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:

*«Разработка компилятора языка В с трансляцией в
машинный код x86-64 под ОС Linux»*

Студент ИУ7-21М
(Группа)

(Подпись, дата)

ИУ7-21М
(И. О. Фамилия)

Руководитель курсовой работы

(Подпись, дата)

Д. А. Салженикин
(И. О. Фамилия)

Консультант

(Подпись, дата)

А. А. Ступников
(И. О. Фамилия)

2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 Аналитическая часть	7
1.1 Назначение разрабатываемого компилятора	7
1.2 Краткая характеристика языка В	7
1.3 Теоретические основы трансляции	8
1.4 Лексические элементы	9
1.5 Требования к компилятору	10
1.6 Вывод	11
2 Конструкторская часть	12
2.1 Общая структура компилятора	12
2.2 Лексический анализ	13
2.3 Таблица символов	14
2.4 Абстрактное синтаксическое дерево	15
2.5 Разбор объявлений	15
2.6 Разбор выражений	16
2.7 Разбор операторов	16
2.8 Генерация x86-64 кода	17
2.9 Вывод	17
3 Технологическая часть	18
3.1 Средства реализации	18
3.2 Особенности реализации лексического анализатора	18
3.3 Особенности реализации синтаксического анализатора	19
3.4 Особенности генерации кода	20
3.5 Пример программы на языке В	20
3.6 Фрагмент полученного ассемблерного кода	21
3.7 Результат выполнения программы	22
3.8 Пример рекурсивной программы	22
3.9 Тестирование	23
3.10 Вывод	24

ЗАКЛЮЧЕНИЕ	25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	27
ПРИЛОЖЕНИЕ А	28

ВВЕДЕНИЕ

Компилятор является одним из ключевых элементов системного программного обеспечения. Он связывает язык высокого уровня с конкретной аппаратной и программной платформой, выполняя лексический анализ, синтаксический разбор, построение внутреннего представления программы и генерацию машинно-зависимого кода. Разработка даже небольшого компилятора позволяет изучить устройство трансляторов, представление выражений и операторов, организацию таблицы символов, соглашения о вызовах функций и особенности целевой архитектуры.

В данной курсовой работе рассматривается язык В. Язык В был разработан Кеном Томпсоном и Деннисом Ритчи в Bell Labs как потомок BCPL и непосредственный предшественник языка С. В историческом варианте В является компактным бестиповым языком, предназначенным прежде всего для системного программирования. Его синтаксис включает внешние определения, функции, автоматические и внешние переменные, векторы, адресную арифметику, условные операторы, циклы, оператор выбора, переходы и развитую систему выражений.

Актуальность работы заключается в том, что язык В достаточно мал для полного описания в рамках курсового проекта, но при этом содержит основные конструкции императивного языка программирования: объявления, области видимости, функции, рекурсию, указатели, массивы, условные и циклические операторы. Поэтому реализация компилятора В позволяет последовательно рассмотреть основные этапы построения транслятора без чрезмерного усложнения грамматики.

Целью курсовой работы является разработка компилятора языка В на языке С, выполняющего трансляцию исходного кода в исполняемый машинный код для архитектуры x86-64 под операционную систему Linux.

Для достижения поставленной цели необходимо решить следующие задачи:

- изучить справочное руководство по языку В и выделить основные синтаксические конструкции языка;
- описать грамматику поддерживаемого подмножества языка В;

- разработать лексический анализатор для выделения ключевых слов, идентификаторов, литералов, операторов и разделителей;
- разработать синтаксический анализатор для объявлений, выражений и операторов языка;
- реализовать таблицу символов для глобальных объектов, локальных переменных, параметров функций, внешних имен и меток;
- построить внутреннее представление программы в виде абстрактного синтаксического дерева;
- реализовать генерацию ассемблерного кода x86-64 для ОС Linux;
- обеспечить сборку исполняемого файла с использованием системного ассемблера и компоновщика;
- подготовить тестовые программы на языке В и проверить соответствие результатов ожидаемому выводу.

Пояснительная записка состоит из аналитической, конструкторской и технологической частей. В аналитической части приведено описание языка В и требований к компилятору. В конструкторской части рассмотрена архитектура транслятора и внутренние структуры данных. В технологической части описана реализация, приведены примеры программ, фрагменты полученного ассемблерного кода и результаты тестирования.

1 Аналитическая часть

1.1 Назначение разрабатываемого компилятора

Разрабатываемый компилятор предназначен для трансляции программ на языке В в ассемблерный код архитектуры x86-64 с последующим получением исполняемого файла под операционную систему Linux. Входом компилятора является один или несколько файлов с исходным текстом программы. Выходом может быть:

- ассемблерный файл с расширением `.s`;
- объектный файл с расширением `.o`;
- исполняемый файл, полученный после компоновки.

Компилятор реализован на языке С. Для сборки промежуточного ассемблерного текста используется системный ассемблер, для компоновки — системный компилятор С, вызываемый как драйвер линковщика. Такой подход позволяет сосредоточить реализацию на трансляции языка В, не разрабатывая собственный ассемблер и компоновщик.

1.2 Краткая характеристика языка В

Согласно руководству Ken Thompson “Users’ Reference to В”, язык В предназначен для рекурсивных, преимущественно нenumерических задач системного программирования [1]. Язык имеет небольшую грамматику, простую модель хранения и богатую систему выражений.

Основные свойства языка В:

- программа состоит из последовательности внешних определений;
- внешнее определение может задавать глобальный объект, вектор или функцию;
- функции объявляются именем, списком параметров и оператором телом;
- локальные автоматические объекты объявляются ключевым словом `auto`;

- внешние имена, используемые внутри функции, объявляются ключевым словом `extrn`;
- язык оперирует машинными словами и исторически не требует строгой системы типов;
- указательная арифметика выражается через операции `*`, `&` и индексирование;
- комментарии записываются в форме `/* ... */`;
- escape-последовательности в строковых и символьных литералах начинаются с символа `*`.

В реализации курсовой работы базовая модель В адаптирована к x86-64. Основной единицей хранения является машинное слово целевой архитектуры. Внешние определения, автоматические объекты, векторы, функции и выражения разбираются в формах, описанных в руководстве по языку В.

1.3 Теоретические основы трансляции

Процесс компиляции можно рассматривать как последовательное преобразование программы из одной формы представления в другую. На первом этапе исходный текст является обычной последовательностью символов, в которой еще не выделены смысловые единицы языка. Лексический анализ группирует символы в токены: ключевые слова, идентификаторы, литералы, операции и разделители. После этого синтаксический анализ проверяет, что последовательность токенов соответствует грамматике языка, и строит внутреннее представление программы. В данной работе таким представлением является абстрактное синтаксическое дерево, поскольку оно сохраняет смысловую структуру программы, но не содержит лишних синтаксических деталей вроде скобок и разделителей.

Абстрактное синтаксическое дерево удобно для компилятора тем, что разные внешние формы записи могут быть сведены к одинаковой внутренней операции. Например, индексирование вектора в языке В выражает адресную арифметику, поэтому выражение обращения к элементу может быть представлено как вычисление адреса и последующее разыменование. Аналогично

составное присваивание сначала разбирается как специальная форма токена, а затем превращается в дерево, в котором отдельно представлены чтение старого значения, бинарная операция и запись результата. Такая нормализация уменьшает количество специальных случаев на этапе генерации кода.

Генерация машинно-зависимого кода требует учета соглашений целевой платформы. Для x86-64 под Linux важны размер машинного слова, способ размещения локальных переменных в стековом кадре, регистры передачи аргументов, регистр возврата результата и правила сохранения регистров при вызовах функций. Компилятор данной работы генерирует ассемблерный текст, а затем передает его системному ассемблеру и компоновщику. Благодаря этому этап трансляции остается достаточно наглядным: результат можно просмотреть как обычный текстовый файл, сопоставить инструкции с исходными конструкциями языка В и отдельно проверить корректность вызовов внешних функций, таких как `printf`.

1.4 Лексические элементы

Лексический анализатор должен выделять из входного потока токены, приведенные в таблице 1.1.

Таблица 1.1 – Основные лексические элементы языка

Группа	Элементы
Идентификаторы	Имена функций, переменных, векторов и меток. Состоят из букв, цифр и символа подчеркивания, не начинаются с цифры.
Ключевые слова В	<code>auto</code> , <code>extrn</code> , <code>if</code> , <code>else</code> , <code>while</code> , <code>switch</code> , <code>case</code> , <code>return</code> , <code>goto</code> .
Литералы	Десятичные и восьмеричные целые константы, символьные константы из одного или двух символов, строковые литералы.
Операторы	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code> , <code>%=</code> , <code>= </code> , <code>=&</code> , <code>=«</code> , <code>=»</code> , <code>==</code> , <code>!=</code> , арифметические, битовые, логические, сравнительные и тернарный оператор.
Разделители	<code>;</code> , <code>,</code> , <code>:</code> , <code>?</code> , круглые, квадратные и фигурные скобки.

Поддерживаемые escape-последовательности языка В приведены в таб-

лице 1.2.

Таблица 1.2 – Escape-последовательности языка В

Последовательность	Значение
*0	Нулевой символ.
*e	Признак конца файла; в реализации представлен значением EOF.
*(, *)	Символы { и }.
*t	Символ табуляции.
**	Символ *.
*', *"	Одинарная и двойная кавычки.
*n	Перевод строки.

1.5 Требования к компилятору

Разрабатываемый компилятор должен выполнять следующие функции:

1. чтение исходного файла на языке В;
2. удаление пробельных символов и комментариев;
3. выделение токенов и проверка корректности литералов;
4. построение таблицы символов;
5. разбор глобальных определений, функций, локальных объявлений и операторов;
6. построение абстрактного синтаксического дерева;
7. проверка корректности операций над машинными словами и адресами;
8. генерация ассемблерного кода x86-64 в синтаксисе GNU assembler;
9. вызов ассемблера и компоновщика для получения исполняемого файла;
10. поддержка режима вывода только ассемблерного файла и режима вывода объектного файла;
11. выполнение набора регрессионных тестов.

1.6 Вывод

В аналитической части рассмотрены назначение компилятора, особенности языка В, теоретические основы трансляции, основные лексические элементы и функциональные требования к транслятору. Грамматика поддерживаемого языка вынесена в приложение А. Реализация соответствует основным конструкциям, описанным в руководстве по В: внешним определениям, функциям, `auto`, `extrn`, выражениям, указательной арифметике, векторам, условным операторам, циклам, `switch`, `goto` и `return`.

2 Конструкторская часть

2.1 Общая структура компилятора

Компилятор построен как последовательность классических этапов трансляции:

1. лексический анализ;
2. синтаксический анализ;
3. построение абстрактного синтаксического дерева;
4. ведение таблицы символов;
5. генерация ассемблерного кода;
6. сборка и компоновка результата.

Внутреннее взаимодействие модулей организовано через общий набор структур, объявленных в заголовочных файлах. Основные глобальные объекты содержат текущий токен, входной и выходной файлы, таблицу символов, параметры генерации кода и флаги командной строки.

Состав модулей компилятора приведен в таблице 2.1.

Таблица 2.1 – Основные модули компилятора

Файл	Назначение
<code>main.c</code>	Обработка параметров командной строки, запуск компиляции, сборки и компоновки.
<code>scan.c</code>	Лексический анализ: чтение символов, удаление комментариев, распознавание литералов, идентификаторов, ключевых слов и операторов.
<code>decl.c</code>	Разбор глобальных и локальных объявлений, функций, параметров, <code>auto</code> и <code>extrn</code> .
<code>stmt.c</code>	Разбор операторов: составных блоков, <code>if</code> , <code>while</code> , <code>switch</code> , <code>case</code> , <code>goto</code> , <code>return</code> , меток и выражений-операторов.

Файл	Назначение
<code>expr.c</code>	Разбор выражений с учетом приоритетов, построение узлов AST для бинарных, унарных, тернарных операций, вызовов функций и индексирования.
<code>tree.c</code>	Создание и отладочная печать узлов абстрактного синтаксического дерева.
<code>sym.c</code>	Таблица символов: поиск и добавление глобальных, локальных, параметрических символов и меток.
<code>types.c</code>	Внутреннее представление машинных слов, адресов и преобразований размеров данных при генерации кода.
<code>gen.c</code>	Машинно-независимый обход AST и выбор операций генерации кода.
<code>cg.c</code>	Машинно-зависимая генерация x86-64 assembly.

2.2 Лексический анализ

Лексический анализатор получает поток символов из входного файла и возвращает токены. При чтении выполняются следующие действия:

- пропускаются пробельные символы;
- удаляются комментарии вида `/* ... */`;
- распознаются односимвольные и многосимвольные операторы;
- числовые литералы, начинающиеся с 0, интерпретируются как восьмеричные;
- строки и символьные литералы обрабатывают escape-последовательности языка C;
- идентификаторы проверяются на совпадение с ключевыми словами.

Для реализации однолексемного просмотра вперед используется механизм возврата токена. Он необходим, например, при разборе оператора, начинающегося с идентификатора: это может быть либо метка вида `name:`, либо выражение.

2.3 Таблица символов

Таблица символов хранит сведения обо всех именах, известных компилятору. Для каждого символа сохраняются:

- имя;
- представление значения;
- структурный вид символа;
- класс хранения;
- размер объекта или метка конца функции;
- смещение локальной переменной или количество параметров функции.

Структурные виды символов:

- `S_VARIABLE` — переменная;
- `S_FUNCTION` — функция;
- `S_ARRAY` — вектор;
- `S_LABEL` — метка;
- `S_EXTERN` — внешнее имя, объявленное через `extrn`.

Классы хранения:

- `C_GLOBAL` — глобальные объекты;
- `C_LOCAL` — локальные автоматические объекты;
- `C_PARAM` — параметры функции.

После завершения генерации тела функции локальные символы удаляются, а глобальные определения остаются доступными для следующих функций.

2.4 Абстрактное синтаксическое дерево

Абстрактное синтаксическое дерево используется как промежуточное представление программы. Узел AST содержит:

- код операции;
- представление результата выражения;
- признак использования в `rvalue`-контексте;
- ссылки на левое, среднее и правое поддеревья;
- целочисленное значение, идентификатор символа или размер.

Три дочерних указателя позволяют единообразно представлять бинарные операции, тернарный оператор, условные операторы и служебные цепочки операторов. Например, для `if` левое поддерево хранит условие, среднее — ветку истинности, правое — ветку `else`. Для тернарного выражения левое поддерево содержит условие, среднее — выражение после `?`, правое — выражение после `:`.

2.5 Разбор объявлений

Глобальные объявления разбираются до конца файла. Если после имени следует `(`, компилятор обрабатывает функцию. Если после имени следует `[`, список инициализаторов или `;`, компилятор обрабатывает внешний объект или вектор.

Историческая форма `B`:

```
name;  
name ival1, ival2;  
name[constant] ival1, ival2;  
name(arg1, arg2) { ... }
```

Локальные объявления `auto` размещаются в стековом кадре функции. Автоматические векторы резервируют последовательность машинных слов, адресуемую через операции индексирования и разыменования. Объявления `extrn` добавляют имя в глобальную таблицу как внешнее до его фактического определения или связывания с внешней функцией.

2.6 Разбор выражений

Выражения разбираются методом приоритетного спуска. Для бинарных операторов задан массив приоритетов. Чем выше значение приоритета, тем раньше оператор связывается с операндами. Унарные и постфиксные операции разбираются отдельно.

Поддерживаются следующие классы выражений:

- целые, символьные и строковые литералы;
- обращение к идентификатору;
- взятие адреса и разыменование;
- индексирование вектора;
- вызов функции по имени или через вычисленное значение;
- префиксный и постфиксный инкремент и декремент;
- арифметические, битовые, логические и сравнительные операции;
- составные присваивания в стиле B;
- тернарный условный оператор.

Индексирование строится через адресную арифметику. Для выражения `v[i]` формируется адрес базы, индекс масштабируется по размеру элемента, затем выполняется сложение и разыменование. Благодаря этому поддерживается свойство B, при котором `v[i]` и `i[v]` эквивалентны.

2.7 Разбор операторов

Операторы языка строятся в AST и затем обходятся генератором кода. Составной оператор `{ ... }` формирует цепочку `A_GLUE`. Оператор `while` содержит поддерево условия и поддерево тела. Оператор `switch` хранит выражение выбора и список `case`-узлов.

Для меток и `goto` используется таблица символов. При первом использовании метка может быть добавлена как еще не определенная. После завершения разбора функции выполняется проверка, что все метки, на которые были переходы, действительно определены.

2.8 Генерация x86-64 кода

Генерация кода разделена на два уровня. Модуль `gen.c` обходит AST и вызывает операции загрузки значений, арифметики, переходов, вызовов функций и сохранения результата. Модуль `cg.c` реализует эти операции через конкретные инструкции x86-64.

Основные решения генератора:

- используется синтаксис GNU assembler;
- глобальные объекты размещаются в секции данных;
- функции помечаются директивой `.globl`;
- локальные переменные размещаются относительно `%rbp`;
- временные значения хранятся в ограниченном наборе регистров;
- вызовы функций используют соглашение System V AMD64 ABI;
- первые аргументы передаются через регистры `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9`;
- результат функции возвращается через `%rax`.

Для оператора `switch` генерируется вспомогательная таблица переходов. Значение выражения сравнивается с константами `case`, после чего управление передается на соответствующую метку или за пределы оператора выбора.

2.9 Вывод

В конструкторской части описана структура компилятора, назначение его модулей, организация таблицы символов, представление AST, разбор объявлений, выражений и операторов, а также принципы генерации x86-64 кода. Выбранная архитектура отделяет машинно-зависимые операции от общего обхода AST, что упрощает сопровождение и проверку транслятора.

3 Технологическая часть

3.1 Средства реализации

Компилятор реализован на языке C. Выбор C обусловлен близостью к системному программированию, удобством работы с файлами, указателями и структурами данных, а также соответствием тематике компилятора исторического языка В [2].

Целевая платформа:

- операционная система — Linux;
- архитектура — x86-64;
- формат промежуточного кода — assembly GNU assembler [3];
- сборка компилятора — `make`;
- сборка результата — `as` и `cc` [4].

Командная строка компилятора поддерживает следующие режимы:

- `-S` — сгенерировать ассемблерный файл и остановиться;
- `-c` — сгенерировать объектный файл и не выполнять компоновку;
- `-o file` — задать имя исполняемого файла;
- `-T` — вывести абстрактное синтаксическое дерево;
- `-v` — вывести команды этапов компиляции.

3.2 Особенности реализации лексического анализатора

Лексический анализатор реализован в файле `scan.c`. Он посимвольно читает входной файл и формирует структуру токена. Для каждого токена сохраняется его тип, строковое описание и целочисленное значение, если токен является литералом или составным присваиванием.

Числовые литералы обрабатываются функцией, которая выбирает основание системы счисления по первому символу. Если литерал начинается с 0, значение интерпретируется как восьмеричное. Это соответствует историческому поведению В.

Строковые и символьные литералы обрабатываются с учетом эскап-последовательностей языка В. Например, строка "%d*n" после лексического анализа содержит символ перевода строки, а не два символа * и n.

3.3 Особенности реализации синтаксического анализатора

Разбор программы начинается с функции обработки глобальных объявлений. Для каждого внешнего определения компилятор определяет его форму по токену, следующему за именем:

- (означает объявление или определение функции;
- [означает определение вектора;
- литерал или имя после глобального объекта означает список инициализаторов;
- ; завершает простое определение.

Операторы разбираются в файле `stmt.c`. Для составных блоков формируется последовательность AST-узлов, связанных операцией `A_GLUE`. Это позволяет генератору кода выполнить операторы в исходном порядке.

Выражения разбираются в файле `expr.c`. Основой является функция разбора бинарных выражений, получающая минимальный допустимый приоритет. Такой способ позволяет корректно разбирать выражения вида:

```
x = a + b * c == d ? f(1) : g(2);
```

Составные присваивания языка В используют форму `x += 1`. Лексер распознает такую запись как присваивание с дополнительной операцией, а парсер преобразует ее в AST, эквивалентный `x = x + 1`.

3.4 Особенности генерации кода

Генератор кода создает текст ассемблерной программы. Для каждой функции формируется пролог:

```
pushq    %rbp
movq     %rsp, %rbp
```

После этого стековый указатель сдвигается на размер области локальных переменных. При выходе из функции выполняется восстановление кадра и инструкция `ret`.

Для локальных переменных используются отрицательные смещения относительно `%rbp`. Для глобальных переменных применяются RIP-relative обращения, например `name(%rip)`. Такой код подходит для современных x86-64 Linux систем.

Вызовы внешних функций выполняются через PLT. Передача аргументов организована в соответствии с соглашением System V AMD64 ABI [5]. Благодаря этому программы на В могут вызывать внешние системные функции, например `printf`, если имя объявлено через `extrn`.

3.5 Пример программы на языке В

Листинг 3.1 — Программа на языке В, демонстрирующая автоматический вектор, указатель и индексирование.

```
main() {
    auto v 4, p;
    extrn printf;

    v[0] = 3;
    v[1] = 4;
    v[2] = 5;
    v[3] = 6;

    p = &v[1];
    printf("%d*n", *p);
}
```

```

    *p = 20;
    printf("%d*n", v[1]);
    printf("%d*n", 2[v]);
    printf("%d*n", *(v + 3));

    return 0;
}

```

В программе создается автоматический вектор `v` из четырех машинных слов и переменная `p`. В переменную `p` записывается адрес элемента `v[1]`. Далее через разыменование изменяется значение этого элемента. Выражения `v[2]`, `2[v]` и `*(v + 3)` демонстрируют адресную модель векторов языка В.

3.6 Фрагмент полученного ассемблерного кода

Листинг 3.2 — Фрагмент ассемблерного кода, полученного для программы из листинга 3.1.

```

main:
    pushq    %rbp
    movq     %rsp, %rbp
    addq     $-48,%rsp
    movq     $3, %r10
    leaq     -32(%rbp), %r11
    movq     $0, %r12
    salq     $3, %r12
    addq     %r11, %r12
    movq     %r10, (%r12)
    leaq     -32(%rbp), %r10
    movq     $1, %r11
    salq     $3, %r11
    addq     %r10, %r11
    movq     %r11, -40(%rbp)
    movq     -40(%rbp), %r10
    movq     (%r10), %r10
    movq     %r10, %rsi

```

```

leaq    L2(%rip), %r10
movq    %r10, %rdi
call    printf@PLT

```

В приведенном фрагменте видно размещение локального вектора в стековом кадре, масштабирование индекса на 8 байт, вычисление адреса элемента и вызов внешней функции `printf`.

3.7 Результат выполнения программы

Листинг 3.3 — Вывод программы из листинга 3.1.

```

4
20
5
6

```

Первое число является исходным значением `v[1]`. Второе число показывает изменение элемента через указатель `p`. Последние две строки демонстрируют корректное чтение элементов вектора через эквивалентные формы индексирования.

3.8 Пример рекурсивной программы

Листинг 3.4 — Рекурсивное вычисление числа Фибоначчи.

```

fib(n) {
    if (n <= 1) return(n);
    return(fib(n - 1) + fib(n - 2));
}

main() {
    extrn printf;

    printf("%d*n", fib(10));
    return(0);
}

```

Листинг 3.5 — Вывод рекурсивной программы.

Этот пример проверяет определение функции, передачу параметров, рекурсивный вызов, условный оператор и возврат значения.

3.9 Тестирование

Для проверки компилятора подготовлен набор тестовых программ в каталоге `tests`. Каждая программа `inputXX.b` имеет соответствующий файл ожидаемого вывода `out.inputXX.b`. Скрипт `tests/runtests` компилирует каждую программу, запускает полученный исполняемый файл и сравнивает фактический вывод с эталонным.

Основные проверяемые возможности приведены в таблице 3.1.

Таблица 3.1 – Проверяемые возможности компилятора

Тесты	Проверяемые конструкции
<code>input01.b</code>	Рекурсия, условный оператор, цикл, арифметика, вызовы функций.
<code>input02.b</code>	Сортировка массивов, вложенные циклы, сравнения, глобальные векторы.
<code>input03.b</code> , <code>input04.b</code>	Обход графа в ширину и глубину, глобальные массивы, вложенные вызовы функций.
<code>input05.b</code>	Моделирование связного списка через массивы индексов.
<code>input06.b</code> – <code>input09.b</code>	Пустые операторы, составные присваивания, отрицательные числа, инициализация глобальных объектов.
<code>input10.b</code> – <code>input16.b</code>	Указатели, адреса элементов вектора, тернарный оператор, <code>goto</code> , <code>switch</code> , вызовы функций через переменную.
<code>input17.b</code> , <code>input18.b</code>	Внешние объявления, глобальные инициализаторы, комментарии и escape-последовательности В.

Команда запуска тестов:

```
make test
```

Пример строки успешного прохождения теста:

input10.b: OK

3.10 Вывод

В технологической части описаны средства реализации, особенности лексического и синтаксического анализа, генерации x86-64 кода, режимы командной строки, примеры программ на языке В, фрагмент полученного ассемблерного кода и подход к регрессионному тестированию.

ЗАКЛЮЧЕНИЕ

В рамках курсовой работы был разработан компилятор языка В на языке С. Компилятор выполняет чтение исходного текста, лексический анализ, синтаксический разбор, построение абстрактного синтаксического дерева, ведение таблицы символов и генерацию ассемблерного кода x86-64 для операционной системы Linux.

В аналитической части была рассмотрена грамматика языка В и выделены его основные конструкции: внешние определения, функции, автоматические и внешние переменные, векторы, адресная арифметика, операторы `if`, `while`, `switch`, `goto`, `return`, выражения и литералы. Также были сформулированы требования к компилятору.

В конструкторской части была разработана структура транслятора. Компилятор разделен на модули лексического анализа, синтаксического анализа объявлений, операторов и выражений, работы с AST, таблицей символов, представлением машинных слов и генерацией кода. Такое разделение позволило изолировать машинно-зависимую часть генератора от общего обхода дерева.

В технологической части была описана реализация компилятора, приведены примеры программ на языке В, фрагмент сгенерированного ассемблерного кода и результаты тестирования. Тестовые программы проверяют рекурсию, циклы, сортировку, обход графов, глобальные и локальные векторы, указатели, тернарный оператор, `switch`, `goto`, внешние объявления, комментарии и `escape`-последовательности.

Таким образом, были решены следующие задачи:

- изучена документация языка В;
- описана грамматика поддерживаемого языка;
- реализован лексический анализатор;
- реализован синтаксический анализатор объявлений, операторов и выражений;
- реализована таблица символов;
- реализовано построение AST;

- реализована генерация x86-64 assembly;
- обеспечена сборка исполняемого файла под Linux;
- подготовлены и описаны регрессионные тесты.

Цель курсовой работы достигнута: разработан компилятор языка В, транслирующий исходные программы в исполняемый машинный код для архитектуры x86-64 под операционную систему Linux.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Thompson K.* Users' Reference to B [Электронный ресурс]. — Режим доступа URL: <https://www.nokia.com/bell-labs/about/dennis-m-ritchie/kbman.html> (Дата обращения: 21.05.2026).
2. *Kernighan B. W., Ritchie D. M.* The C Programming Language. — Prentice Hall, 1988.
3. GNU Assembler Manual [Электронный ресурс]. — Режим доступа URL: <https://sourceware.org/binutils/docs/as/> (Дата обращения: 21.05.2026).
4. GCC, the GNU Compiler Collection [Электронный ресурс]. — Режим доступа URL: <https://gcc.gnu.org/> (Дата обращения: 21.05.2026).
5. System V Application Binary Interface. AMD64 Architecture Processor Supplement [Электронный ресурс]. — Режим доступа URL: <https://gitlab.com/x86-psABIs/x86-64-ABI> (Дата обращения: 21.05.2026).

ПРИЛОЖЕНИЕ А

Ниже приведена укрупненная грамматика поддерживаемого ядра языка В. Запись не повторяет дословно историческую нотацию руководства, а адаптирует ее к реализации компилятора.

Листинг А.1 — Укрупненная грамматика внешних определений.

```
program      ::= { definition }

definition   ::= name [ "[" [ constant ] "]" ] [ initializer-list ] ";"
               | name "(" [ parameter-list ] ")" compound-statement

initializer-list ::= initializer { "," initializer }
initializer      ::= constant | name
parameter-list   ::= parameter { "," parameter }
parameter        ::= name
```

Листинг А.2 — Грамматика операторов.

```
statement ::= "auto" auto-list ";"
           | "extrn" name-list ";"
           | name ":" statement
           | "case" constant ":" statement
           | "{" { statement } "}"
           | "if" "(" expression ")" statement [ "else" statement ]
           | "while" "(" expression ")" statement
           | "switch" expression statement
           | "goto" expression ";"
           | "return" [ "(" expression ")" | expression ] ";"
           | [ expression ] ";"

auto-list  ::= auto-decl { "," auto-decl }
auto-decl  ::= name [ constant | "[" [ constant ] "]" ]
name-list  ::= name { "," name }
```

Листинг А.3 — Грамматика выражений.

```

expression ::= assignment
assignment ::= conditional [ assign-op assignment ]
conditional ::= binary-expression
               | binary-expression "?" expression ":" conditional

primary     ::= name
               | constant
               | string
               | "(" expression ")"

postfix     ::= primary { "(" [ argument-list ] ")"
               | "[" expression "]"
               | "++"
               | "--" }

prefix      ::= postfix
               | "&" prefix
               | "*" prefix
               | "-" prefix
               | "!" prefix
               | "~" prefix
               | "++" prefix
               | "--" prefix

binary-expression ::= prefix { binary-op prefix }

```

При синтаксическом анализе выражений используется таблица приоритетов операторов. Присваивание является правоассоциативным. Тернарный оператор также связывается справа налево. Остальные бинарные операторы обрабатываются с учетом их приоритета и левой ассоциативности.